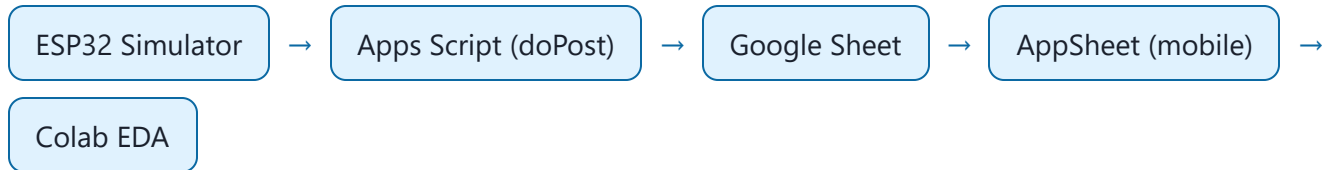


Activity 3 — Build Your Own Smart Mushroom Farm Mobile App

A complete IoT monitoring system: a sensor **simulator** sends data to a **Google Apps Script**, which stores it in **Google Sheets**, displayed as a live mobile app in **AppSheet** — and analyzed in **Colab**.



Prerequisites

- Google account (free), Chrome browser, a smartphone for testing.
- Simulator: <https://setcertification.com/datascience/baguio/session5/> (also [index2.html](#) and [index3.html](#) for the other simulators).

Part 1 — Create your Google Sheet

1. Google Drive → + **New** → **Google Sheets**.
2. Bold + color Row 1 (headers get created automatically by the script).
3. Copy the **Sheet ID** from the URL: `.../spreadsheets/d/[THIS_IS_YOUR_SHEET_ID]/edit` — save it.

Part 2 — Google Apps Script (the backend)

1. Go to `script.google.com` → **New Project** → name it `MushroomFarmAPI_[YourName]`.
2. Delete the default code and paste the script below.
3. Replace `YOUR_SHEET_ID_HERE` with your Sheet ID, then **Save**.
4. **Deploy** → **New deployment** → **Web app**; Execute as **Me**; Who has access **Anyone**; **Deploy**; authorize.
5. Copy the **Web App URL** (ends with `/exec`) — you'll paste it into the simulator.

This one script **auto-detects** the simulator type (Mushroom Farm, Weather Station, Tourism Monitor, Forest Watch) and sets the right headers automatically.

APPS SCRIPT (CODE.GS)

```

// REPLACE WITH YOUR GOOGLE SHEET ID
const SHEET_ID = 'YOUR_SHEET_ID_HERE';
const SHEET_NAME = 'Sheet1';

function doPost(e) {
  try {
    let data;
    if (e.postData && e.postData.contents) {
      try { data = JSON.parse(e.postData.contents); }
      catch (jsonError) { data = e.parameter; }
    } else { data = e.parameter; }

    const sheet = SpreadsheetApp.openById(SHEET_ID).getSheetByName(SHEET_NAME);

    if (sheet.getLastRow() === 0) {
      let headers = [];
      if (data.temperature !== undefined && data.humidity !== undefined) {
        if (data.soilMoisture !== undefined) {
          headers =
['Timestamp','DeviceID','Temperature','Humidity','Light','CO2','SoilMoisture'];
        } else if (data.pressure !== undefined) {
          headers =
['Timestamp','StationID','Province','Temperature','Humidity','Pressure','WindSpeed','Rainfall','UV
Index'];
        }
      } else if (data.localTourists !== undefined) {
        headers =
['Timestamp','MonitorID','Province','LocalTourists','ForeignTourists','GroupTours','TotalVisitors'
,'HotelOccupancy','TimePeriod','TopDestination'];
      } else if (data.treeCount !== undefined) {
        headers =
['Timestamp','MonitorID','Forest','Temperature','Moisture','FireRisk','AirQuality','Rainfall','Gro
wthRate','TreeCount','Coverage','Species','HealthStatus'];
      } else {
        headers = ['Timestamp', ...Object.keys(data)];
      }
      sheet.getRange(1, 1, 1, headers.length).setValues([headers]);
      sheet.getRange(1, 1, 1,
headers.length).setFontWeight('bold').setBackground('#4a5568').setFontColor('#ffffff');
    }

    let row = [new Date()];
    if (data.soilMoisture !== undefined) {
      row.push(data.deviceId||'Unknown', data.temperature||0, data.humidity||0, data.light||0,
data.co2||0, data.soilMoisture||0);
    } else if (data.pressure !== undefined) {

```

```

        row.push(data.stationId||'Unknown', data.province||'Unknown', data.temperature||0,
data.humidity||0, data.pressure||0, data.windSpeed||0, data.rainfall||0, data.uvIndex||0);
    } else if (data.localTourists !== undefined) {
        row.push(data.monitorId||'Unknown', data.province||'Unknown', data.localTourists||0,
data.foreignTourists||0, data.groupTours||0, data.totalVisitors||0, data.hotelOccupancy||0,
data.timePeriod||'Unknown', data.topDestination||'Unknown');
    } else if (data.treeCount !== undefined) {
        row.push(data.monitorId||'Unknown', data.forest||'Unknown', data.temperature||0,
data.moisture||0, data.fireRisk||0, data.airQuality||0, data.rainfall||0, data.growthRate||0,
data.treeCount||0, data.coverage||0, data.species||0, data.healthStatus||'Unknown');
    } else {
        Object.keys(data).forEach(key => { if (key !== 'timestamp') row.push(data[key]); });
    }

    sheet.appendRow(row);
    if (sheet.getLastRow() === 2) sheet.autoResizeColumns(1, sheet.getLastColumn());
    if (sheet.getLastRow() > 10000) sheet.deleteRow(2);

    return ContentService.createTextOutput(JSON.stringify({
        'status':'success','message':'Data logged successfully',
        'simulator':detectSimulatorType(data),'timestamp':new Date().toISOString()
    })).setMimeType(ContentService.MimeType.JSON);
} catch (error) {
    return
    ContentService.createTextOutput(JSON.stringify({'status':'error','message':error.toString()}))
        .setMimeType(ContentService.MimeType.JSON);
}
}

function detectSimulatorType(data) {
    if (data.soilMoisture !== undefined) return 'Mushroom Farm';
    if (data.pressure !== undefined) return 'Weather Station';
    if (data.localTourists !== undefined) return 'Tourism Monitor';
    if (data.treeCount !== undefined) return 'Forest Watch';
    return 'Unknown';
}

function doGet(e) {
    try {
        const sheet = SpreadsheetApp.openById(SHEET_ID).getSheetByName(SHEET_NAME);
        const lastRow = sheet.getLastRow();
        if (lastRow > 1) {
            const headers = sheet.getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0];
            const data = sheet.getRange(lastRow, 1, 1, sheet.getLastColumn()).getValues()[0];
            const response = {};
            headers.forEach((header, index) => { response[header] = data[index]; });

```

```

        return
        ContentService.createTextOutput(JSON.stringify(response)).setMimeType(ContentService.MimeType.JSON
    );
    }
    return ContentService.createTextOutput(JSON.stringify({'message':'No data
available'})).setMimeType(ContentService.MimeType.JSON);
    } catch (error) {
        return
        ContentService.createTextOutput(JSON.stringify({'error':error.toString()})).setMimeType(ContentSer
vice.MimeType.JSON);
    }
}

```

Part 3 — Connect the simulator

1. Open the simulator URL in Chrome (see Prerequisites).
2. In the Control Center, paste your **Web App URL**; set a Device ID; choose Manual / Auto / Stress.
3. Click **Start Simulation** (or **Send Once** to test). Check your Sheet — rows should appear.

No data? Confirm the Web App URL is correct and ends with /exec. Try **Send Once** first.

Part 4 — Build the AppSheet mobile app

1. appsheet.com → sign in → **Create** → **App** → **Start with existing data** → choose your Sheet.
2. Customize: app name **Smart Mushroom Farm Monitor**, theme green, add a logo.
3. Add views: **Dashboard**, **Current Conditions** (detail of latest row), **Temperature Trend** (line chart), **All Readings** (table sorted by Timestamp desc).
4. Format rules: Temperature 18–24 green, outside red. Add a **Status** virtual column (formula below).
5. Automation → Bots: alert when [Temperature] > 28.

APPSHEET — STATUS VIRTUAL COLUMN

```

IF(AND([Temperature]>=18, [Temperature]<=24, [Humidity]>=80, [Humidity]<=95),
    "✅ Optimal", "⚠️ Check Conditions")

```

APPSHEET — OPTIMAL RANGE INDICATOR

```
IF(AND([Temperature]>=18,[Temperature]<=24,[Humidity]>=80,[Humidity]<=95), "🟢",  
IF(OR([Temperature]<15,[Temperature]>30,[Humidity]<60), "🔴", "🟡"))
```

Part 5 — Analyze in Colab

Open the **Activity 3 Colab notebook**. Set SHEET_NAME to your sheet's name and run it — the script auto-detects the simulator type and produces overview stats, time-series and correlation charts, anomaly detection, insights, and an enhanced CSV export. It works for all four simulators.